OCR détection et reconnaissance de texte

OCR : détection et reconnaissance de texte

Définition et cas d'usage OCR

L'**OCR** (*Optical Character Recognition*) désigne l'ensemble des méthodes permettant de **convertir le texte présent dans une image** (photo, scan, capture d'écran) en **texte numérique exploitable**.

On distingue souvent :

- **Détection** : localiser les zones de texte (lignes, mots, caractères) dans l'image.
- **Reconnaissance** : transcrire le contenu textuel des zones détectées.

Cas d'usage fréquents :

- **Numérisation de documents** : factures, formulaires, pièces d'identité, contrats.
- **Industrie / logistique** : lecture d'étiquettes, numéros de série, plaques.
- **Retail** : lecture de prix/étiquettes, inventaires, tickets de caisse.
- **Accessibilité** : lecture à haute voix, aide à la navigation.
- **Vision "in-the-wild"** : texte dans la rue, panneaux, enseignes, sous-titres.

Spécificités OCR (vs classification "classique") :

- Le texte a une **structure séquentielle** (caractères et mots) : lien fort avec le NLP.
- La variabilité est grande : **polices**, **rotations**, **déformations**, **éclairage**, **flou**, **fonds complexes**.
- La performance se mesure souvent **au niveau du texte** (erreurs de transcription), pas uniquement au niveau image.

OCR "document" vs OCR "scene text"

On rencontre souvent deux grandes familles de problèmes :

- **OCR de documents** (scan, PDF rasterisé) :
  - Texte généralement aligné, fonds plus propres.

    - Les défis viennent surtout de la **qualité du scan**, des tableaux, des tampons, des signatures.
    - Les objectifs incluent souvent l'extraction structurée (champs : date, montant, TVA, n° facture).
- **OCR en environnement réel** (*scene text*) :
    - Texte dans des photos : **panneaux**, **enseignes**, **étiquettes**, reflets.
    - Beaucoup plus de variations géométriques (rotation, perspective) et photométriques (ombres, flou).
    - La détection devient souvent la partie la plus difficile.

Pourquoi l'OCR est un problème "Vision ↔ NLP"

Même si l'entrée est une image, la sortie est une **séquence**. Cela implique :

- Des modèles capables de produire des séquences (CTC, attention, Transformers).
- Un intérêt fort pour des post-traitements inspirés du NLP :
    - **modèles de langue** (prior sur la probabilité des mots)
    - **dictionnaires** métier
    - normalisation (formatage)

Erreurs typiques (pour guider l'analyse)

- Confusions visuelles : `O/0`, `I/l/1`, `S/5`, `B/8`.
- Erreurs de segmentation : caractères collés, mots coupés.
- Erreurs géométriques : texte incliné, perspective (un caractère "écrasé").
- Bruit de fond : motifs, textures, faible contraste.

Métriques spécifiques (CER, WER)

Les métriques OCR évaluent l'écart entre un texte prédit et un texte de référence.

CER — *Character Error Rate*

Le **CER** mesure le taux d'erreur au niveau des caractères.

On utilise la distance d'édition (Levenshtein) :

```
CER = (S + D + I) / N
```

où :

- `S` = substitutions de caractères
- `D` = suppressions (*deletions*)
- `I` = insertions
- `N` = nombre de caractères dans la référence

Interprétation :

- **0%** = transcription parfaite
- Plus le CER est bas, mieux c'est

WER — *Word Error Rate*

Le **WER** applique la même logique mais au niveau des mots :

```
WER = (S + D + I) / N
```

où `N` est ici le nombre de mots de la référence.

Quand utiliser quoi :

- **CER** : plus stable pour des langues/écritures où la segmentation en mots est délicate, ou pour du texte court.
- **WER** : plus parlant pour des documents orientés "lecture" (phrases), mais sensible à la tokenisation (ponctuation, apostrophes, etc.).

Exemple rapide (CER)

Référence : `FACTURE 2026`

Prédiction : `FACTURF 2026`

Ici on a une substitution (`E` remplacé par `F`) :

- `S = 1`, `D = 0`, `I = 0`
- `N = 12` (en comptant l'espace)

Donc :

```
CER = 1 / 12 ≈ 0,083
```

Exemple rapide (WER)

Référence : `TOTAL TTC 123,45`

Prédiction : `TOTAL TCC 123,45`

Au niveau des mots :

- Une substitution : `TTC` → `TCC`
- `N = 3`

```
WER = 1 / 3 ≈ 0,333
```

Ce que CER/WER ne capturent pas toujours

- **Impact métier** : confondre `0` et `O` peut être critique sur un identifiant, mais marginal sur une phrase.
- **Champs structurés** : sur une date `12/01/2026`, une seule erreur peut invalider tout le champ.

En pratique on ajoute souvent :

- un taux de champs valides (regex / règles)
- des métriques par sous-ensembles (chiffres uniquement, montants, codes)
- des courbes précision/rappel sur la détection

Chaîne OCR complète

Une chaîne OCR moderne se décompose souvent ainsi :

1. **Acquisition**
   - Photo / scan / flux vidéo
2. **Prétraitements** (améliorer la lisibilité)
   - Redressement (*deskew*), correction de perspective
   - Débruitage, amélioration de contraste
   - Binarisation (parfois), *super-resolution* (selon cas)
3. **Détection de texte** (où est le texte ?)
   - Sortie typique : boîtes englobantes (rectangles) ou polygones, parfois avec score
4. **Regroupement et ordonnancement**
   - Regrouper en lignes/paragraphes
   - Définir l'ordre de lecture (gauche→droite, haut→bas, multi-colonnes)
5. **Rectification des crops**
   - Redresser une zone courbe ou inclinée avant reconnaissance

6. **Reconnaissance** (quel texte ?)
   - Modèle séquentiel (CNN+RNN/Transformer) ou classification caractère par caractère
7. **Post-traitements**
   - Dictionnaire / lexique métier
   - Correction orthographique
   - Contraintes de format (IBAN, date, TVA, n° facture)
   - *Confidence scoring* et règles de rejet
8. **Validation**
   - Contrôles métier
   - *Human-in-the-loop* sur les cas incertains

Points clés qui dominent la performance :

- **Qualité de détection** (une mauvaise boîte détériore la reconnaissance)
- **Normalisation géométrique** (rotation/perspective)
- **Domaine** (polices spécialisées, champs structurés, écritures manuscrites)

Zoom sur quelques étapes critiques

Prétraitements : quand et pourquoi

Les prétraitements sont utiles quand ils réduisent la variabilité inutile :

- **Correction de perspective** : photos de documents (smartphone) où les lignes ne sont plus horizontales.
- **Deskew** : scan légèrement incliné (quelques degrés suffisent à faire baisser la reconnaissance).
- **Contraste** : texte gris clair sur fond clair.

Attention : trop de prétraitements “agressifs” peuvent supprimer de l’information (ex : binarisation mal paramétrée).

Ordre de lecture

L’OCR ne s’arrête pas à “reconnaître des boîtes”. Il faut souvent reconstruire une lecture :

- documents multi-colonnes
- tableaux
- champs dispersés (factures)

Deux approches classiques :

- heuristiques (tri par `y` puis par `x`, regroupements en lignes)
- modèles dédiés (layout analysis) quand la mise en page est complexe

Post-traitements

Une grande part de la qualité perçue vient des post-traitements :

- **lexiques** : liste de produits, villes, codes
- **formats** : dates, montants, IBAN
- **validation** : score de confiance + règles

L'idée centrale : transformer un "texte probable" en "texte valide" selon ton contexte.

Méthodes de détection

Objectif : produire des régions de texte robustes, même quand le texte est petit, incliné, sur fond bruité.

Notions préliminaire

NMS (Non-Maximum Suppression)

NMS est un post-traitement standard en détection d'objets/texte.

But : lorsqu'un détecteur produit **plusieurs boîtes** qui se recouvrent pour le **même** mot/ligne, on veut ne garder que la meilleure.

Principe (idée) :

1. On trie les boîtes par **score de confiance** décroissant.
2. On garde la meilleure boîte.
3. On supprime les boîtes qui se recouvrent trop avec elle (chevauchement mesuré via **IoU**, *Intersection over Union*) au-dessus d'un seuil.
4. On répète.

Réglage clé : le **seuil NMS**.

- Seuil faible → on supprime agressivement → risque de perdre des boîtes proches (rappel ↓).
- Seuil élevé → on supprime peu → risque de doublons (précision ↓) et post-traitement plus

difficile.

EAST

EAST (*Efficient and Accurate Scene Text detector*) est un détecteur de texte “dense” :

- Il produit une **carte** de présence de texte et des **paramètres géométriques** pour reconstruire des boîtes.
- Points forts :
  - Efficace en inférence (souvent utilisé en production)
  - Simple à intégrer
- Limites :
  - Gère moins bien le texte **très courbe**
  - La qualité dépend fortement des seuils et du post-traitement (NMS)

Sortie typique : rectangles (parfois orientés) + score.

Intuition : détection dense vs détection par ancres

EAST est proche d’un détecteur “dense” : au lieu de proposer explicitement des régions candidates (propositions), il prédit localement :

- “y a-t-il du texte ici ?”
- “si oui, à quoi ressemble la boîte autour ?”

Cela donne une inférence souvent efficace, mais impose un bon post-traitement.

Points de réglage importants

- **Seuil de score** : trop haut → perte de rappel (on rate du texte), trop bas → faux positifs.
- **NMS** : contrôle le chevauchement accepté entre boîtes.
- **Taille d’entrée** : si le texte est petit, une entrée trop petite va le faire disparaître après sous-échantillonnage.

Erreurs typiques côté détection

- Une seule grande boîte qui englobe plusieurs lignes (mauvais regroupement).
- Boîtes tronquées (le dernier caractère hors boîte).
- Faux positifs sur motifs répétitifs (grilles, textures, briques).

CRAFT

CRAFT (*Character Region Awareness for Text detection*) modélise le texte à un niveau plus fin :

- Il estime des cartes de probabilité pour :
  - régions de caractères
  - affinités entre caractères (pour regrouper en mots/lignes)
- Points forts :
  - Très bon pour du texte **irrégulier**, **densité variable**, et peut mieux gérer le “texte naturel”
  - Permet un regroupement plus précis
- Limites :
  - Plus coûteux
  - Le post-traitement (regroupement) est plus complexe

Sortie typique : polygones ou boîtes orientées issues du regroupement de régions.

Intuition : “caractères + affinités”

CRAFT s’appuie sur une idée simple :

- on repère d’abord des **régions de caractères**
- puis on estime si deux régions sont “liées” (affinité)

En regroupant ces régions, on reconstitue mots/lignes. Cette granularité aide quand le texte est irrégulier (espacements variables, polices atypiques).

Post-traitement (idée générale)

Sans rentrer dans l’implémentation exacte, on fait typiquement :

- seuillage des cartes (caractère / affinité)
- extraction de composantes connexes
- fusion en polygones/boîtes

Les seuils ont un impact direct :

- seuil trop faible → fusion excessive (mots collés)
- seuil trop fort → fragmentation (un mot coupé en morceaux)

Choisir entre EAST et CRAFT

Critères pratiques :

- **Si tu veux une solution simple et rapide (prod-friendly)** : plutôt **EAST**.
  - Cas typiques : documents assez “propres”, texte plutôt rectiligne, besoin d’inférence rapide.
  - Points d’attention : réglages **score threshold** + **NMS** + taille d’entrée.
- **Si tu as du texte irrégulier / dense / “scene text” difficile** : plutôt **CRAFT**.
  - Cas typiques : texte petit, spacements variables, polices très variées, texte naturel.
  - Points d’attention : post-traitement plus complexe (seuils régions/affinités), coût plus élevé.

Règle empirique :

- EAST peut rater du texte difficile (ou produire des boîtes trop grossières) mais est souvent excellent pour démarrer.
- CRAFT peut mieux “accrocher” du texte irrégulier, au prix d’un pipeline plus délicat à régler.

Méthodes de reconnaissance

Une fois la zone de texte extraite (crop), la reconnaissance consiste à prédire une **séquence de caractères**.

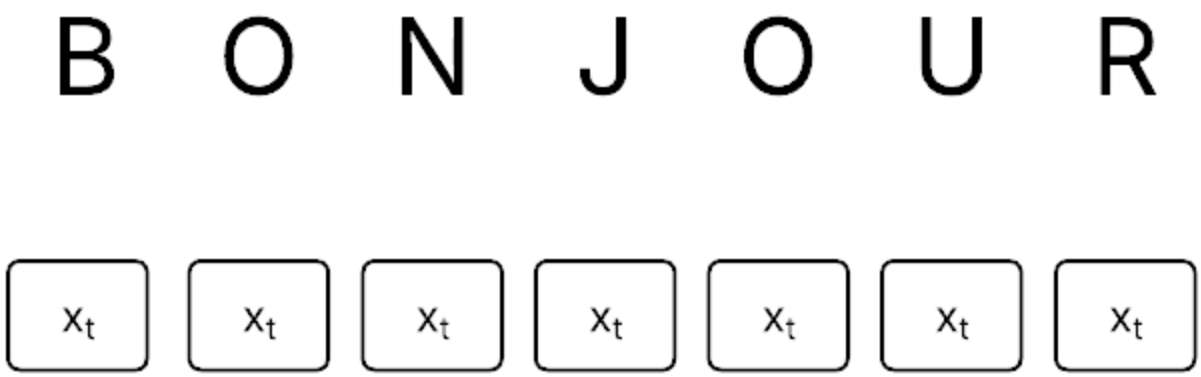
Le problème

Dans une image de texte, une fois que le CNN a extrait des **features visuelles**, on obtient souvent une **séquence de vecteurs** le long de l’axe horizontal (de gauche à droite).

- Chaque “colonne” de features voit **une partie du caractère**, mais **pas le mot entier**.

Exemple :

```
B  O  N  J  O  U  R
|  |  |  |  |  |  |
```



Un caractère dépend souvent :

- de ce qu'il y a **avant** (ex : `rn` vs `m`)
- de ce qu'il y a **après** (`l` vs `I`)

CRNN

CRNN (*Convolutional Recurrent Neural Network*) est un classique de la reconnaissance OCR "end-to-end" :

- **CNN** : extrait des features visuelles du mot/ligne (caractéristiques locales)
- **RNN** (souvent BiLSTM) : modélise la dépendance séquentielle le long de l'axe horizontal
- **CTC** (*Connectionist Temporal Classification*) : permet d'entraîner sans alignement explicite caractère↔position

Rappel : Recurrent Network

Un **RNN** traite une séquence **pas à pas**, en gardant une **mémoire** du contexte précédent.

Idée centrale : à chaque pas de temps t, le modèle combine :

- l'entrée courante x_t (ex : la colonne de features CNN à la position t)
- un **état caché** h_{t-1} qui résume l'information accumulée jusque-là

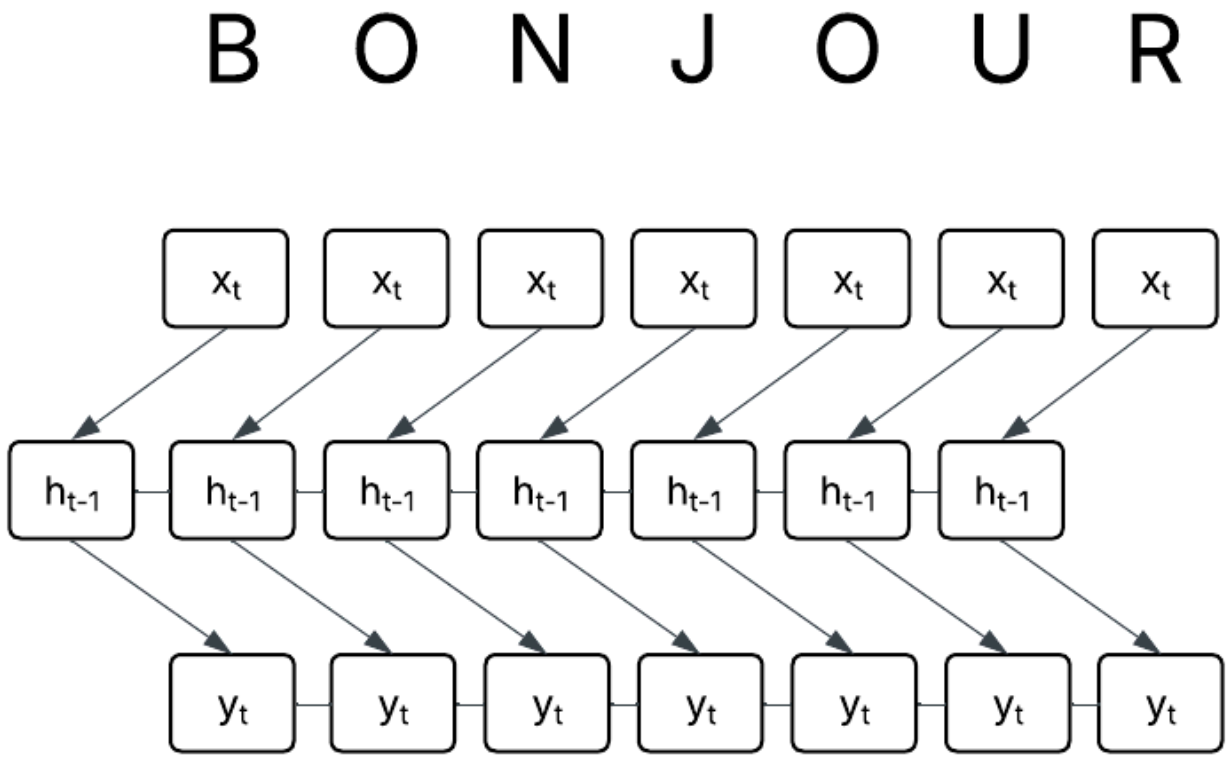
pour produire un nouvel état h_t et une sortie (probabilités de caractères).

On peut résumer par :

$$h_t = f(x_t, h_{t-1})$$

$$y_t = g(h_t)$$

Ce mécanisme permet d'introduire du **contexte** : une décision à la position `t` dépend aussi de ce qui a été vu avant.



Limites des RNN "simples" :

- Difficulté à apprendre des dépendances longues (problèmes de gradients qui disparaissent/explosent)
- Sensibilité au bruit : si le contexte est mal "mémorisé", la prédiction peut dériver

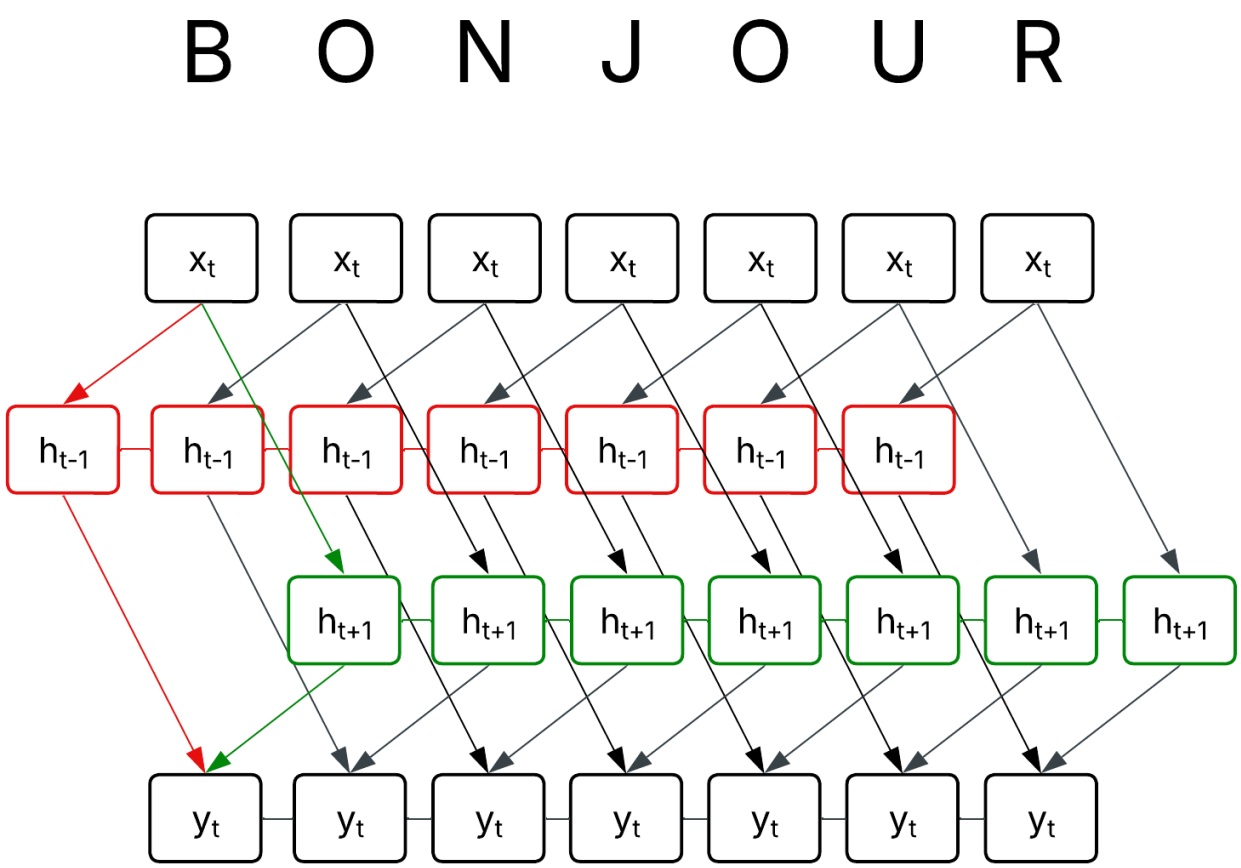
---

Pourquoi **BiLSTM** ?

Un **BiLSTM** lit la séquence :

- **de gauche → droite**
- **de droite → gauche**

-> Chaque position voit **le passé ET le futur**.



Donc pour un caractère ambigu :

- le contexte gauche aide
- le contexte droit aide aussi

Résultat : bien meilleur pour le texte que des LSTM simples.

“le long de l’axe horizontal”

Concrètement :

- l’image → CNN → feature map
- on **aplati la hauteur**
- chaque colonne = un pas de temps
- Le BiLSTM traite la séquence de colonnes

CTC (Connectionist Temporal Classification)

Le problème clé

On veut entraîner le modèle avec :

- une **image de mot**
- sa **transcription texte**

Mais on ne sait pas :

- quel caractère correspond à quelle colonne
- combien de colonnes pour chaque caractère

Pas d'alignement caractère ↔ position.

La solution CTC

CTC permet d'entraîner le modèle **sans alignement explicite**.

Il introduit :

- un symbole spécial : **blank (∅)**

Le réseau prédit à chaque pas :

```
[a, b, c, ..., blank]
```

Exemple

Cible : `BON`

Une prédiction possible :

`![[Embedding - CTC.png]] B B ∅ O ∅ N N ∅`

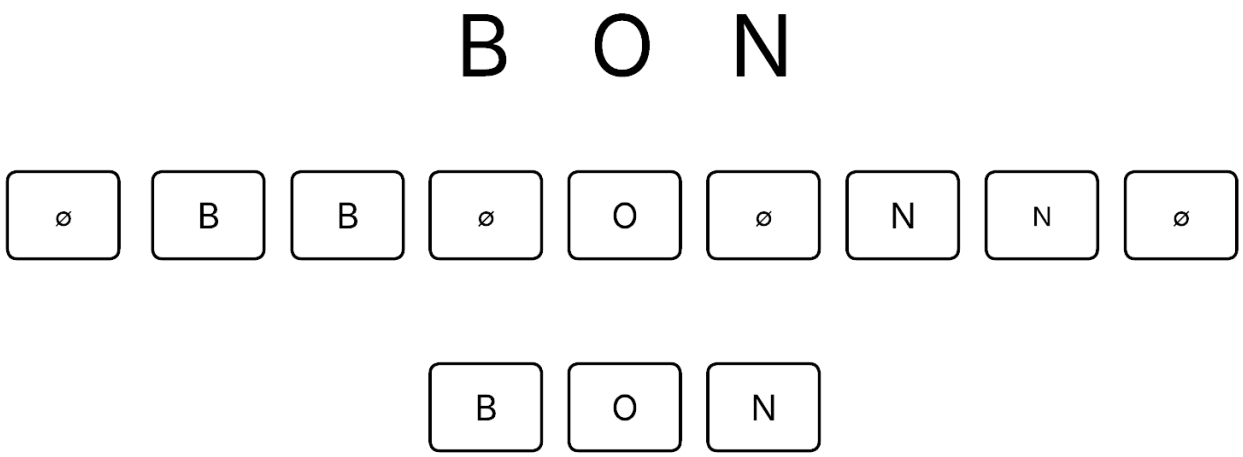
CTC applique deux règles :

1. **Supprimer les répétitions consécutives**
2. **Supprimer les blanks**

Résultat :

`BON`

Alignement trouvé automatiquement !



Comment ça s’entraîne ?

CTC :

- considère **tous les alignements possibles** compatibles avec le mot
- calcule la **probabilité totale**
- maximise cette probabilité (via dynamic programming)

-> Le réseau apprend **où** et **quand** produire chaque caractère.

Pourquoi RNN + CTC vont ensemble ?

RNN / BiLSTM	CTC
Modélise le contexte séquentiel	Gère l’absence d’alignement
Prédictions dépendantes	Apprentissage flexible
Donne une séquence riche	Transforme en texte final

-> **BiLSTM produit une séquence intelligente**,
-> **CTC la transforme en mot sans alignement manuel**.

En résumé

- **BiLSTM** : comprend le texte comme une phrase, pas caractère isolé
- **CTC** : apprend à aligner automatiquement image ↔ texte
- Ensemble = standard des systèmes OCR modernes

Entrée du modèle : normalisation des crops

Avant d'envoyer un crop au CRNN, on cherche une représentation stable :

- redressement (rotation), parfois rectification
- conversion niveaux de gris (souvent)
- normalisation (moyenne/écart-type)

Deux erreurs classiques :

- **écraser** la largeur (aspect ratio) → lettres déformées
- crop trop serré → amputations de caractères

Sortie du modèle : décodage CTC

En pratique, deux stratégies :

- **Greedy** : on prend le caractère le plus probable à chaque pas, puis on compacte (rapide).
- **Beam search** : on garde plusieurs hypothèses (meilleur, mais plus lent).

Le beam search devient vraiment utile quand :

- les mots sont longs
- le texte est bruité
- on combine avec un **lexique** ou un **modèle de langue**

Où le CRNN échoue souvent

- texte très petit (résolution insuffisante)
- texte courbe ou fortement perspectif sans rectification
- alphabets très larges (beaucoup de symboles), si le dataset est petit

Données et annotation

Pour entraîner un CRNN, on a typiquement besoin de paires :

- image de mot/ligne
- transcription correspondante

Points d’attention :

- cohérence du vocabulaire (mêmes caractères autorisés)
- gestion des caractères rares
- nettoyage des labels (un label bruité peut pénaliser fortement)